

Parallel Project Report: Evaluation of SpMV under Different OpenMP Scheduling Strategies and Computational Resources

Francesco Brunelli (ID: 245252)

Email: francesco.brunelli@studenti.unitn.it

Course: Introduction to Parallel Computing (2025-2026)

Abstract—Sparse Matrix Vector Multiplication (SpMV) is a computationally challenging problem due to its dependence on both the structure of the input matrix and the hardware characteristics. This work evaluates the performance of dynamic, guided and static OpenMP scheduling strategies across different thread counts and memory configurations using five matrices from the SuiteSparse collection. The experiments show that the highest performance improvements are not always achieved with the largest number of threads or the most memory. Static scheduling consistently achieves the best speedup despite its lack of adaptability, while dynamic and guided policies at default settings pay an elevated overhead cost. These results highlight the importance of choosing appropriate scheduling strategies based on matrix properties and computational resources.

Index Terms—Sparse Matrix-Vector Multiplication (SpMV), Compressed Sparse Row (CSR), COO, OpenMP (OMP)

I. INTRODUCTION

Sparse Matrix Vector Multiplication (SpMV) is a computationally challenging problem due to its dependence on both the structure of the input matrix and the characteristics of the underlying computational architecture [1]. While many studies evaluate OpenMP scheduling for SpMV, most focus on fixed thread counts or ignore the interaction between scheduling granularity, NUMA effects, and varying memory budgets. This paper provides a concise overview of the performance achievable by different scheduling techniques under varying combinations of computational resources and degrees of parallelism, using the OpenMP (OMP) API. Since sparse matrices, by definition, contain very few non-zero (NNZ) elements, it is reasonable to convert their representation into formats containing only non-zero elements in order to reduce memory and computational needs [2]. Two of the most commonly used representations are the Coordinate (COO) format and the Compressed Sparse Row (CSR) format. All matrices used in this study were obtained from the SuiteSparse Matrix Collection [6] in MatrixMarket format, and thus were initially provided in COO representation. The main contributions of this work are the performance characterization of SpMV under different OpenMP scheduling strategies, an evaluation of their scalability across multiple parallel configurations and different matrices, and a comparison against a sequential baseline. The results illustrate how scheduling choices interact with matrix

structure and available computational resources, offering practical insight into selecting appropriate parallelization strategies for SpMV workloads. The remainder of this paper is organized as follows: Section II introduces the COO and CSR formats and provides an overview of the three scheduling strategies adopted in this work. Section III describes the experimental methodology including the code structure and toolchain used for data collection. Section IV presents and discusses the obtained performance results. Finally, Section V summarizes the main conclusions.

II. BACKGROUND

This section provides the background necessary to understand the sparse matrix formats (COO and CSR), and the three OpenMP scheduling strategies used in this study.

A. COO Sparse Matrix Format

The matrices used for this study were initially provided in COO format via the SuiteSparse Matrix Collection. In COO, each non-zero element is represented by its value along with its row and column indices, stored in separate arrays [3]. While simple, COO is less efficient in SpMV computation, so all matrices were converted to CSR format for the experiments.

B. CSR Sparse Matrix Format

The Compressed Sparse Row (CSR) format is a widely used sparse matrix representation. Like COO, it uses three arrays: one for non-zero values, one for column indices and a third one for the starting index of each row's non zero elements [3]. This structure enables more efficient memory access during SpMV computations.

C. OpenMP Scheduling Strategies Adopted

OpenMP (OMP) is a shared memory parallel programming standard that is based on the fork/join parallel execute model [5]. This work focuses on three OMP scheduling strategies: static, dynamic and guided (Figure 1).

- **Static:** Iterations are divided into chunks of a specified size and assigned to threads in a round-robin manner. Choosing an appropriate chunksize is critical to ensure balanced workloads and minimize load imbalance across threads [4].
- **Dynamic:** Iterations are broken up into chunks, and each thread executes one chunk at a time. When a thread

finishes a chunk, it requests another one from the runtime system until all iterations are completed. If the chunk size is omitted, a default value of 1 is used; however, this may increase scheduling overhead for large iteration counts [4].

- **Guided:** Similar to dynamic scheduling, but chunksize decreases as the execution progresses. If no chunk size is specified, the size of the chunks decreases to 1; otherwise, they decrease to the specified size. This approach can improve load balance when later iterations are more computationally intensive [4].

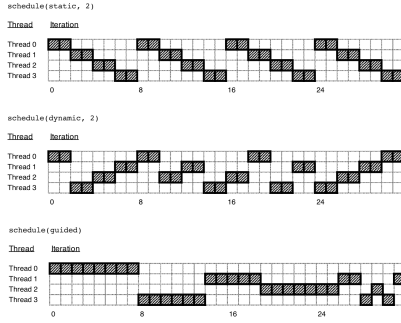


Fig. 1: [4]

III. METHODOLOGY

This section describes the methods and code used and the criteria adopted to gather the necessary data for this work.

A. The SpMV computation flow

The SpMV evaluation pipeline consists of three stages: (i) reading the matrix in COO format from a MatrixMarket file, (ii) converting it to the CSR representation, and (iii) executing the SpMV kernel either sequentially or using OpenMP.

1) *MatrixMarket Parsing and COO Loading:* MatrixMarket files encode the numerical type (real, complex, integer, pattern) and the structural properties of the matrix (general or symmetric). General matrices are loaded directly into COO format. For symmetric matrices, only the upper or lower triangular part is stored, so the missing symmetric entries are replicated during loading. Pattern matrices do not include numerical values; therefore, a default value (e.g., 1.0) is assigned to each NNZ entry during the loading process.

2) *Conversion from COO to CSR format:* Although COO is convenient for loading, it is inefficient for SpMV due to irregular memory access patterns. For this reason, the COO matrix is converted into CSR which enables continuous row access and improves cache locality. The conversion computes the row-pointer array by counting the number of NNZ entries in each row. The resulting CSR structure is then used throughout the SpMV experiments.

3) *SpMV computation:* Once the CSR structure is built, SpMV is then computed using the standard row-wise kernel shown in Algorithm 1. A random vector of dimension equal to the number of matrix columns is generated for each experiment.

Algorithm 1

```

procedure CSRMUL( $CSR(aRow, aCol, aVal), v$ )
  Output:  $out$ 
  for  $i \leftarrow 0$  to  $size(aRow) - 1$  do
     $temp \leftarrow 0$ 
    for  $j \leftarrow aRow[i]$  to  $aRow[i + 1] - 1$  do
       $temp \leftarrow temp + aVal[j] * v[aCol[j]]$ 
    end for
     $out[i] \leftarrow temp$ 
  end for
end procedure

```

B. Parallelization of SpMV (OpenMP)

The parallel version of the SpMV kernel has been implemented by adding the OMP directive: `#pragma omp parallel for schedule(<scheduling_type>)` before the outer loop of Algorithm 1. This allows the iterations to be distributed across multiple threads. Choosing an appropriate scheduling strategy (static, dynamic or guided) ensures load balancing and efficient use of computational resources. The OpenMP implementation preserves sequential correctness while improving performance on multi-core systems.

C. Experimental Automation Scripts

To streamline data collection, two categories of scripts were developed:

1) **Bash scripts:** automate job submission and execution for different number of threads (from 1 to 64 in powers of 2), partially avoiding manual executions and walltime overruns.

2) **PBS scripts:** manage job submission to the cluster's compute nodes, collects the outputs, and organizes results into structured folders for subsequent analysis.

These scripts automatically gather metrics such as execution time, cache usage (via cachegrind), and performance statistics (via perf) for all combinations of threads, memory available and scheduling strategies.

IV. EXPERIMENTS AND SYSTEM DESCRIPTION

A. Experimental Setup

All experiments were executed on a multi-socket x86_64 server equipped with four Intel Xeon Gold 6140M processors (18 cores per socket, 72 cores in total) and 4 NUMA nodes. The operating system is CentOS Linux 7, and GCC 4.8.5 was used for compilation with flags `-O3 -march=native -fopenmp` (Table I).

B. Metrics

Time: execution time of CSR x V multiplication, expressed in milliseconds (ms); 90 percentile of 100 runs

Cache Stats: measured with perf and cachegrind; reported as percentages; average of 5 runs

(Strong) Scaling: fixed size, different number of threads

Speedup: compared to sequential execution

TABLE I: Hardware and Software Configuration

Component	Specification
Architecture	x86_64
CPU Model	Intel(R) Xeon(R) Gold 6140M CPU @ 2.30GHz
Sockets	4
Cores per socket	18
Total cores	72
Threads per core	1
NUMA nodes	4
L1 cache	32 KB (I) + 32 KB (D) per core
L2 cache	1 MB per core
L3 cache	25 MB per socket
Operating System	CentOS Linux 7 (Core)
Compiler	GCC 4.8.5 (-O3 -march=native -fopenmp)

C. Dataset

We use a set of five matrices from the SuiteSparse Collection [6] (Table II).

TABLE II: Dataset used for the experiments

Graph	Matrix Name	Rows	Columns	Non-zeros	NNZ [%]
	fd12	7 500	7 500	28 462	0.05
	sinc12	7 500	7 500	283 992	0.51
	epb1	14 724	14 724	95 053	0.04
	c-46	14 913	14 913	130 397	0.06
	human_gene2	14 340	14 340	18 068 388	8.79

V. RESULTS AND DISCUSSION

In this section, we evaluate the performance of multi-threaded SpMV under different scheduling policies, thread counts, memory budgets and matrix characteristics.

A. Execution time

All execution time measurements were repeated 100 times, and the 90th percentile was used to mitigate outliers. Throughout all thread-scaling plots, the point at 1 thread corresponds to sequential execution.

1) *Overall Performance*: Increasing the number of threads does not necessarily result in a proportional reduction in the execution time. When the number of threads increases, the overhead of thread management and synchronization may outweigh the benefits of parallelism, particularly for matrices with a relatively small number of NNZ elements. As a result, parallel execution may be slower than sequential execution, as shown in Figure 2(a). Similarly, providing additional memory does not always improve performance. Figure 2(b) shows that, in some cases, performance even worsens at higher thread counts. This occurs because in a NUMA architecture, although the operating system typically attempts to keep memory accesses local to each processor, as the data size grows (resulting from the ability to load larger portions of the matrix into memory), there may be instances where remote memory access becomes necessary. This can lead to a decrease in the application’s memory bandwidth and a subsequent degradation in the program’s performance [8]. For denser matrices with

significantly more NNZ elements, the parallelization overhead becomes less dominant. In these cases, increasing the number of threads will result in a significant speedup of the execution time. Larger memory allocations further help by enabling bigger chunks in dynamic and guided scheduling without excessive cache misses, as shown in Figure 3(b), where execution time decreases with increasing threads and memory.

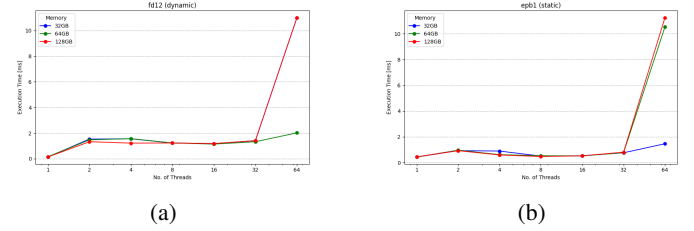


Fig. 2

2) *The scheduling impact*: Three OpenMP scheduling strategies were evaluated:

- Static (chunksize = 100)
- Dynamic (default chunksize = 1)
- Guided (default minimum chunksize = 1)

Figures [3(a)-(c)] compare the behaviour of these policies. Dynamic and guided scheduling approaches benefit from larger memory allocations when processing highly NNZ-dense matrices, as more memory reduces pressure on cache and improves load balancing. However, with very small matrices or matrices with low NNZ elements, the overhead of managing very small chunks (often of size 1) degrades performance, making both dynamic and guided slower than static scheduling. Static scheduling behaves differently: with a fixed chunk size the execution time is relatively stable with respect to memory size as long as the working set fits into cache; for very large matrices exceeding cache capacity, memory-related factors can still influence performance. Figure 3(d) clearly depicts how static scheduling can scale effectively up to a point, after which the trade-off between a higher level of parallelism and synchronization overhead results in a slower execution time.

B. (Strong) Scaling

The trends observed in the previous subsection are confirmed by the strong-scaling plots. For small matrices (e.g., Figure 4 (a)), performance worsens as the number of threads increases independently from the scheduling policy, due to the overhead that outweighs the benefits of parallelization. As noted in [7], “when using small matrices the static scheduling delivers better performance levels than dynamic and guided schedulings. This is attributed to the fact that the execution times of small matrices are short and dynamic and guided schedulings require some overheads for handling the threads”. For larger matrices, such as human_gene2 (Figure 4 (b)), static scheduling again provides the highest speedup. This is due to the fact that dynamic and guided scheduling policies suffer from substantial overhead because of their chunksize of 1,

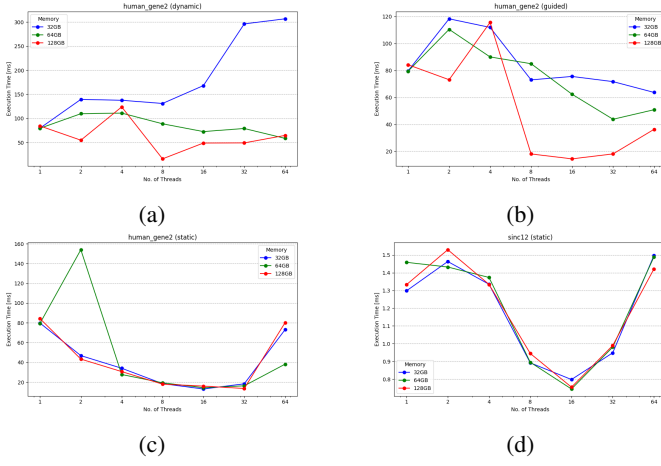


Fig. 3

which generates a high frequency of scheduling events. Guided scheduling progressively reduces chunksize toward 1 as the number of left NNZ elements reduces, which further increases the overhead in the tail of the execution. Dynamic scheduling with a chunksize of 1 provides the highest flexibility and the best load balancing; however, it also introduces the largest overhead due to frequent work requests. Static scheduling, despite the lack of adaptability, avoids such overhead and therefore outperforms guided and dynamic policies in most of the cases.

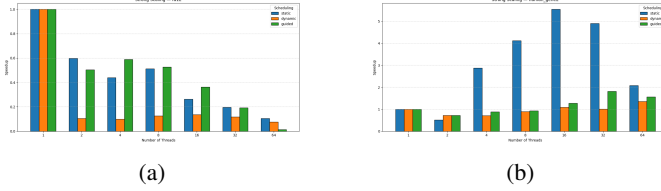


Fig. 4: Threads = 1 refers to sequential execution

C. Cache performance

Figure 5 shows the cache-miss percentages for all scheduling strategies using 8 threads. Overall, L1 miss rates remain low ($\approx 4 - 6\%$), while LLC miss rates are more informative:

- **Dynamic scheduling** exhibits the highest cache-miss rate, particularly at low thread counts. This is due to its extremely fine granularity (chunksize = 1), which causes poor spatial locality.
- **Guided scheduling** generally produces the lowest LLC-miss percentages. The first iterations use relatively large chunks, which allow threads to access continuous data sequentially, preserving spatial locality. An exception occurs for the very large human_gene2 matrix, where the initial chunk sizes exceed cache capacity.
- **Static scheduling** tends to have higher miss rates than guided scheduling because its fixed, relatively small chunks (100 elements) break spatial locality.

As shown by Figure 6, increasing the number of threads reduces cache-miss percentages for all strategies. This is primarily because each thread operates on a smaller portion of the matrix and the input vector, improving cache utilization per thread and reducing pressure on shared caches. However, for very large matrices, LLC misses remain significant due to the working set exceeding cache capacity, regardless of the scheduling.

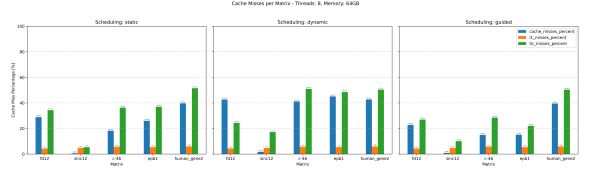


Fig. 5

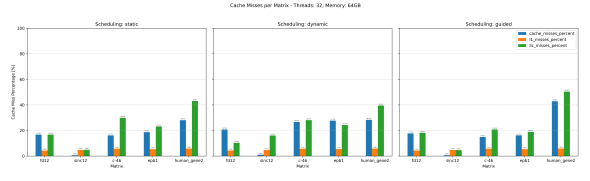


Fig. 6

VI. CONCLUSIONS

In this study, we evaluated the performance of Sparse Matrix-Vector Multiplication (SpMV) on a multi-socket x86_64 server using five different matrices and multiple OpenMP scheduling strategies under different thread counts and memory configurations. The results show that the highest performance improvements are not always achieved with the largest number of threads or the most memory. Among the evaluated strategies, static scheduling consistently provided the highest speedup despite its lack of adaptability. In contrast, dynamic and guided scheduling with small default chunk sizes incurred significant overhead, leading to longer execution times in some configurations. From a cache-performance perspective, dynamic scheduling showed the highest LLC-cache misses, while guided scheduling generally achieved the lowest. Increasing the number of threads improved the cache utilization per thread, reducing the amount of cache misses for all strategies. Overall, the main performance bottlenecks arise from scheduling overhead (especially with chunksize = 1), NUMA-induced memory bandwidth limitations, and cache-capacity effects for large matrices. These results provide insights into how scheduling strategies, matrix characteristics and computational resources interact in parallel SpMV. Future work could explore performance differences between a shared memory implementation (as in this study) and a distributed memory approach.

VII. REPRODUCIBILITY & ARTIFACT AVAILABILITY

All code, datasets, and plots used in this study are publicly available at the following GitHub repository: [9].

REFERENCES

- [1] K. Hou, W. -c. Feng and S. Che, "Auto-Tuning Strategies for Parallelizing Sparse Matrix-Vector (SpMV) Multiplication on Multi- and Many-Core Processors," 2017 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), Lake Buena Vista, FL, USA, 2017, pp. 713-722, doi: 10.1109/IPDPSW.2017.155. keywords: Kernel;Sparse matrices;Graphics processing units;Computational modeling;Systems architecture;Programming;SpMV;GPU;APU;AMD;Machine Learning;auto-tuning;performance tuning;CSR;binning;classification;graph processing.
- [2] A. A. K. Al-Saeedi, S. O. Yurievna and T. W. Khairi, "Improving the efficiency of sparse matrix class processing by using the SPM-CSR parallel algorithm and OpenMP technology," 2022 4th International Youth Conference on Radio Electronics, Electrical and Power Engineering (REEPE), Moscow, Russian Federation, 2022, pp. 1-4, doi: 10.1109/REEPE53907.2022.9731504. keywords: Power engineering;Instruction sets;Software algorithms;Systems architecture;Software;Sparse matrices;Parallel algorithms;OpenMP;SpMV;reduction;sparse matrix;CSR.
- [3] N. Kasmi, S. A. Mahmoudi, M. Zbakh and P. Manneback, "Performance evaluation of sparse matrix-vector product (SpMV) computation on GPU architecture," 2014 Second World Conference on Complex Systems (WCCS), Agadir, Morocco, 2014, pp. 23-27, doi: 10.1109/ICoCS.2014.7060964. keywords: Graphics processing units;Equations;Clocks;Instruction sets;Programming;Arrays;Sparse matrix;SpMV;GPU;Cusp;MKL;CUDA;Sparse Matrix Vector Product.
- [4] Peter S. Pacheco, Matthew Malensek, An Introduction to Parallel Programming, Second Edition, Morgan Kaufmann, 2021. ISBN: 978-0-12-804605-0
- [5] S. Liu, Y. Zhang, X. Sun and R. Qiu, "Performance Evaluation of Multithreaded Sparse Matrix-Vector Multiplication Using OpenMP," 2009 11th IEEE International Conference on High Performance Computing and Communications, Seoul, Korea (South), 2009, pp. 659-665, doi: 10.1109/HPCC.2009.75. keywords: Sparse matrices;Job shop scheduling;Kernel;High performance computing;Processor scheduling;Libraries;Scalability;Packaging;Software packages;Multicore processing;OpenMP;Multithreaded;Sparse Matrix-Vector Multiplication;Load Balance.
- [6] Kolodziej et al., (2019). The SuiteSparse Matrix Collection Website Interface. Journal of Open Source Software, 4(35), 1244, <https://doi.org/10.21105/joss.01244> Available online at: <https://sparse.tamu.edu/>
- [7] S. Ohshima, T. Katagiri and M. Matsumoto, "Performance Optimization of SpMV Using CRS Format by Considering OpenMP Scheduling on CPUs and MIC," 2014 IEEE 8th International Symposium on Embedded Multicore/Manycore SoCs, Aizu-Wakamatsu, Japan, 2014, pp. 253-260, doi: 10.1109/MCSoC.2014.43. keywords: Dynamic scheduling;Sparse matrices;Microwave integrated circuits;Hardware;Libraries;Schedules;Graphics processing units;Open MP;Auto-tuning;SpMV;OpenATLib;Xabclib.
- [8] G. Xiao, Z. Li, Y. Chen, Y. Zeng and W. Yang, "NESpMV: NEON-Enhanced SpMV Algorithm for NUMA Architectures," 2024 IEEE Smart World Congress (SWC), Nadi, Fiji, 2024, pp. 2477-2484, doi: 10.1109/SWC62898.2024.00374. keywords: Jacobian matrices;Neon;Single instruction multiple data;Instruction sets;Memory architecture;Vectors;Mathematical models;Registers;Sparse matrices;Iterative methods;ARM Processors;SpMV;NUMA;Sparse matrix storage formats;NEON.
- [9] <https://github.com/FrancescoBrunelli/PARCO-Computing-2026-245252.git>